Deploying an AI-Parrot Agent to Microsoft Teams & Copilot Studio

Step-by-step guide for exposing an AI-Parrot agent through the Microsoft 365 Agents SDK, making it available in **MS Teams** and/or **Microsoft Copilot Studio**.

Table of Contents

---

1. Prerequisites

- Access to the **navigator-agent-server** repository: https://github.com/Trocdigital/navigator-agent-server
- An Azure subscription with permissions to create **Bot Services** and **App Registrations** (or access to someone who can — see Step 3).
- The `parrot` CLI installed (`pip install ai-parrot` or `uv pip install ai-parrot`).
- The server where the agent will be deployed must be reachable via HTTPS (e.g. behind Cloudflare Tunnel, ngrok, or a public load balancer).

---

2. Create the Agent Code

Step 1 — Clone the **navigator-agent-server** repo and create a branch for your agent:

```
git checkout -b feat/my-new-agent
```

Step 2 — Create the agent directory under `agents/`, or copy an existing one as a starting point:

```
# From scratch
mkdir agents/my_agent

# Or copy an existing agent
cp -r agents/porygon agents/my_agent
```

Step 3 — At a minimum, the agent directory needs:

- An agent definition YAML (or Python file) that registers the bot with `BotManager`.
- A system prompt or instructions file.
- Any custom tools the agent needs.

Step 4 — Register the agent's `chatbot_id` in the agent server's config so `BotManager.get_bot()` can resolve it at runtime.

---

3. Create Azure Bot + App Registration

> This step requires Azure AD admin privileges. At TROC, the App Registration permissions must be approved by **Llorente** (Azure AD admin).

3.1 Create the App Registration

1. Go to **Azure Portal** > **Azure Active Directory** > **App registrations**

   > **New registration**.

2. Set the name (e.g. `my-agent-bot`).
3. Select **Single tenant** (recommended).
4. Click **Register**.

5. Note the following values:
6. **Application (client) ID** → this becomes `{ID}_MICROSOFT_APP_ID`
7. **Directory (tenant) ID** → this becomes `{ID}_MICROSOFT_TENANT_ID`
8. Go to **Certificates & secrets** > **New client secret** > copy the secret value → this becomes `{ID}_MICROSOFT_APP_PASSWORD`

3.2 Create the Azure Bot Service

1. Go to **Azure Portal** > **Create a resource** > search **Azure Bot**.
2. Fill in:
3. **Bot handle**: e.g. `my-agent-bot`
4. **Pricing tier**: F0 (free) for dev, S1 for production.
5. **Microsoft App ID**: select "Use existing app registration" and paste the App ID from step 3.1.
6. **App type**: Single Tenant.
7. **App tenant ID**: paste the tenant ID from step 3.1.
8. Click **Create**.

> **Do not configure the messaging endpoint yet** — wait until the agent is deployed (Step 7).

---

4. Generate Security Keys

Use the `parrot generate-keys` CLI command to create the API key and HMAC secret. The prefix must match the agent's name as declared in `integrations_bots.yaml` (uppercased).

```
# Generate and display keys
parrot generate-keys --prefix MY_AGENT

# Or generate and write directly to a .env file
parrot generate-keys --prefix MY_AGENT --write .env
```

This produces two values:

Variable	Purpose	Format
`{ID}_API_KEY`	Inbound API-Key authentication. Used by Copilot Studio's direct connector (sends `x-api-key` header). Azure channels (Teams, Web Chat, DirectLine) use Bot Framework JWT instead.	URL-safe base64, 48 random bytes
`{ID}_HMAC_SECRET`	HMAC-signed request authentication for the A2A companion surface. Only needed if you configure `kind: a2a` separately or plan to use HMAC auth.	Hex-encoded, 32 random bytes

Example

```
$ parrot generate-keys --prefix PORYGON

PORYGON_API_KEY=8Z3op0GmLGIhnq2JuPSwTAokV3zGO2Lt2lsKOvGoxvXNXnSDTF6wmxt6yBLG7X2s
PORYGON_HMAC_SECRET=30ba2f7b5e1fa05d3c1f55327e53262c2e5220d46d8da86731628a70e8c3c450

Copy these into your .env file, or re-run with --write .env
```

> **Note**: `{ID}_HMAC_SECRET` is only used by the A2A companion surface when `kind: a2a` is configured with `hmac_secret`. For `kind: msagent` (the standard Teams/Copilot path), only `{ID}_API_KEY` is actively used for inbound auth.

---

5. Configure `integrations_bots.yaml`

Add your agent to `env/integrations_bots.yaml`. This file tells the integration manager how to wire the bot at startup.

```
agents:
  MyAgent:                          # Agent name (env var prefix =
MY_AGENT)
    kind: msagent                   # MS Agent SDK integration
    chatbot_id: my_agent            # Must match the bot ID in
BotManager

    # --- Azure AD (resolved from env vars by convention) ---
    # MY_AGENT_MICROSOFT_APP_ID, MY_AGENT_MICROSOFT_APP_PASSWORD,
    # MY_AGENT_MICROSOFT_TENANT_ID are read automatically.
    app_type: SingleTenant

    # --- Inbound API-Key auth (for Copilot Studio) ---
    # Resolved from MY_AGENT_API_KEY env var.
    api_key_header: x-api-key       # Must match the header in
Copilot Studio

    # --- Messaging endpoint ---
    # Default: /api/msagentsdk/myagent/messages
    # Set to /api/messages if Copilot Studio requires a fixed path:
    endpoint: /api/messages

    # --- A2A companion surface (always on with kind: msagent) ---
    url: https://my-agent.agents.trocdigital.io
    tags:
      - my-agent
      - customer-support

    # --- Optional: O365 OAuth2 SSO/OBO ---
    # o365_client_id: ...
    # o365_client_secret: ...
    # o365_tenant_id: ...
    # redirect_uri: https://my-agent.agents.trocdigital.io/auth/
callback

    # --- Optional: Bot Framework OAuth connections ---
    # oauth_connections:
    #   o365: graph_sso
    # obo_scopes:
    #   o365:
    #     - https://graph.microsoft.com/.default

    welcome_message: "Hello! I'm MyAgent. How can I help you?"
    debug: false                    # Set to true for verbose logging
```

Key Fields

Field	Required	Description
`kind`	Yes	Must be `msagent` for Teams/Copilot integration.
`chatbot_id`	Yes	The bot's ID as registered with `BotManager`.
`app_type`	No	`SingleTenant` (default) or `MultiTenant`.
`api_key_header`	No	Header name for API key auth (default: `x-api-key`).
`endpoint`	No	Custom messaging route. Set to `/api/messages` for Copilot Studio compatibility.
`url`	No	Public base URL for the A2A companion AgentCard.
`tags`	No	Tags surfaced in the AgentCard for A2A discovery.
`welcome_message`	No	Sent when a new user joins the conversation.
`oauth_connections`	No	Map of tool name → Azure Bot OAuth connection name for user-delegated tokens.

---

6. Hand Off Credentials to Platform Team

Collect all credentials and hand them to the **platform/infra team** (Oslan) for injection into the deployment environment.

Credential Checklist

```
# Azure AD (from App Registration — Step 3)
{ID}_MICROSOFT_APP_ID=<Application (client) ID>
{ID}_MICROSOFT_APP_PASSWORD=<Client secret value>
{ID}_MICROSOFT_TENANT_ID=<Directory (tenant) ID>

# Security keys (from parrot generate-keys — Step 4)
{ID}_API_KEY=<generated API key>
{ID}_HMAC_SECRET=<generated HMAC secret>
```

Replace `{ID}` with the uppercased agent name as declared in the YAML (e.g. `PORYGON`, `MY_AGENT`, `CONCIERGE`).

> **Security**: Never commit these values to the repository. They must be injected via environment variables, secrets manager, or `.env` file on the server.

---

7. Post-Deployment: Azure Bot Configuration

Once the platform team confirms the agent is deployed and accessible via HTTPS:

7.1 Set the Messaging Endpoint

1. Go to **Azure Portal** > **Bot Services** > your bot > **Configuration**.
2. In the **Messaging endpoint** field, enter the appropriate URL (see below).
3. Click **Apply**.

Messaging endpoint URL (per-bot canonical route):

```
https://{SERVER_DOMAIN}/api/msagentsdk/{agent_name}/messages
```

Or, if you set `endpoint: /api/messages` in the YAML:

```
https://{SERVER_DOMAIN}/api/messages
```

Example:

```
https://porygon.agents.trocdigital.io/api/msagentsdk/porygon/messages
```

7.2 Enable MS Teams Channel

1. Go to **Channels** in the Azure Bot.
2. Click **Microsoft Teams** > **Apply**.
3. Accept the Terms of Service.
4. The channel should show as **Running**.

7.3 Test in Web Chat

1. In the Azure Bot, click **Test in Web Chat**.
2. Send a message — you should get a response from your agent.

3. If you get a 401/403 error, verify the Azure AD credentials are correct in the environment.

7.4 Test in Teams

1. In **Channels**, next to the **Microsoft Teams** channel, click **Open in Teams**.
2. Teams will open with a direct chat with the bot.
3. Send a message and verify the response.

---

8. Post-Deployment: Microsoft Copilot Studio

To make the agent available as a Copilot Studio agent (for richer orchestration, custom topics, or embedding in other Copilot surfaces):

8.1 Create a Copilot Studio Agent

1. Go to https://copilotstudio.microsoft.com/.
2. Click **Agents** > **+ New agent** > **Create blank agent**.
3. Configure the agent name, description, and instructions.

8.2 Connect to the AI-Parrot Agent

1. In the Copilot Studio agent, go to the **Agents** tab.
2. Click **+ Add an agent**.
3. Select the connection type (see URL options below).
4. Configure authentication:
5. Auth type: **API Key**
6. Header name: `x-api-key`
7. API Key value: the `{ID}_API_KEY` value from Step 4.
8. Click **Add** and test the connection.

Connection URL — MS Agent SDK protocol (recommended, full Bot Framework support):

```
https://{SERVER_DOMAIN}/api/msagentsdk/{agent_name}/messages
```

Connection URL — A2A protocol (lightweight, text-only — covers ~80% of use cases):

```
https://{SERVER_DOMAIN}/.well-known/agent-card.json
```

MS Agent SDK vs A2A Protocol

Aspect	MS Agent SDK	A2A Protocol
URL	`/api/msagentsdk/{name}/messages`	`/.well-known/agent-card.json`
Content	Text + Adaptive Cards + Attachments	Text only
Auth (inbound)	Bot Framework JWT + API Key	API Key / JWT / HMAC
OAuth/SSO	Full support (OBO, token exchange)	Not supported
Best for	Teams, Copilot Studio, rich UI	Agent-to-agent communication

---

9. Testing

Quick Smoke Test

```
# Test the A2A discovery endpoint
curl -s https://{SERVER_DOMAIN}/.well-known/agent-card.json | python
-m json.tool

# Test the A2A directory (lists all agents)
curl -s https://{SERVER_DOMAIN}/a2a/directory | python -m json.tool

# Test the messaging endpoint (requires API key)
curl -X POST https://{SERVER_DOMAIN}/api/msagentsdk/{agent_name}/
messages \
  -H "Content-Type: application/json" \
  -H "x-api-key: {YOUR_API_KEY}" \
  -d '{"type": "message", "text": "Hello"}'
```

Test via Bot Framework Emulator

1. Download the Bot Framework Emulator.
2. Connect to:
   `https://{SERVER_DOMAIN}/api/msagentsdk/{agent_name}/messages`
3. Enter the Microsoft App ID and Password.
4. Send a test message.

---

10. URL Reference

All URLs below assume the server is at `https://{SERVER_DOMAIN}` and the agent name (lowercased) is `{agent_name}`.

Bot Framework / Copilot Studio

Endpoint	URL
Per-bot messaging (canonical)	`POST /api/msagentsdk/{agent_name}/messages`
Fixed messaging (Copilot Studio)	`POST /api/messages`

A2A Companion Surface

Endpoint	URL
Agent Card (v1.0)	`GET /.well-known/agent-card.json`
Agent Card (v0.3)	`GET /.well-known/agent.json`
Agent directory	`GET /a2a/directory`
Send message	`POST /a2a/{agent_name}/message/send`
Stream message	`POST /a2a/{agent_name}/message/stream`
List tasks	`GET /a2a/{agent_name}/tasks`
Get task	`GET /a2a/{agent_name}/tasks/{task_id}`
Cancel task	`POST /a2a/{agent_name}/tasks/{task_id}/cancel`
JSON-RPC	`POST /a2a/{agent_name}/rpc`

Auth Exclusions

The following paths are excluded from the navigator session/ABAC auth chain (they authenticate themselves via Bot Framework JWT or API key):

- `/api/msagentsdk/*`
- `/api/messages`
- `/a2a` and `/a2a/*`
- `/.well-known/*`

11. Environment Variable Reference

All variables use the uppercased agent name as prefix (e.g. `PORYGON`, `CONCIERGE`).

Variable	Source	Required	Description
`{ID}_MICROSOFT_APP_ID`	Azure App Registration	Yes	Application (client) ID
`{ID}_MICROSOFT_APP_PASSWORD`	Azure App Registration	Yes	Client secret
`{ID}_MICROSOFT_TENANT_ID`	Azure App Registration	Yes	Directory (tenant) ID
`{ID}_API_KEY`	`parrot generate-keys`	Recommended	API key for inbound auth (Copilot Studio)
`{ID}_HMAC_SECRET`	`parrot generate-keys`	Optional	HMAC secret for A2A companion auth
`{ID}_MICROSOFT_APP_TYPE`	Manual	No	`SingleTenant` or `MultiTenant` (default: `SingleTenant`)
`{ID}_ENDPOINT`	Manual	No	Custom messaging route override
`{ID}_JWT_SECRET`	Manual	No	JWT secret for A2A companion auth
`{ID}_O365_CLIENT_ID`	Azure App Registration	No	O365 OAuth2 SSO client ID
`{ID}_O365_CLIENT_SECRET`	Azure App Registration	No	O365 OAuth2 SSO client secret
`{ID}_O365_TENANT_ID`	Azure App Registration	No	O365 OAuth2 SSO tenant ID
`{ID}_REDIRECT_URI`	Manual	No	OAuth2 redirect URI

12. Troubleshooting

401 Unauthorized on messaging endpoint

- **Bot Framework JWT issue**: Verify `{ID}_MICROSOFT_APP_ID`, `{ID}_MICROSOFT_APP_PASSWORD`, and `{ID}_MICROSOFT_TENANT_ID` are correct in the environment.
- **API Key issue**: Check that `{ID}_API_KEY` is set and matches what Copilot Studio sends. The wrapper logs `"API-Key validation failed (header=x-api-key)"` on mismatch.
- **Multi-tenant vs Single-tenant**: If the bot uses `MultiTenant`, the outbound token must be minted against `botframework.com`, not the home tenant. Set `app_type: MultiTenant` in the YAML.

404 Not Found

- Verify the messaging endpoint URL in Azure Bot Configuration matches the actual route.
- If using `endpoint: /api/messages`, make sure it's set in the YAML — the wrapper only registers this route when explicitly configured.

Bot replies with empty messages or errors

- Check the agent logs for exceptions (set `debug: true` in YAML).
- Verify `chatbot_id` in the YAML matches a registered bot in `BotManager`.
- Ensure the agent server can reach external APIs (LLM providers, databases).

Copilot Studio shows "Agent unavailable"

- Verify the server is reachable from the internet (HTTPS required).
- Test the endpoint manually with curl (see Step 9).
- Check that the API key in Copilot Studio matches `{ID}_API_KEY`.

A2A discovery returns empty

- The `/.well-known/agent-card.json` route is registered by the first `kind: a2a` or `kind: msagent` bot that starts. If no bots are configured, the route won't exist.
- Check `/a2a/directory` — it lists all registered AgentCards.

Appendix: Full Example (`integrations_bots.yaml`)

```
agents:
  Porygon:
    kind: msagent
    chatbot_id: porygon
    app_type: SingleTenant
    api_key_header: x-api-key
    endpoint: /api/messages
    url: https://porygon.agents.trocdigital.io
    tags:
      - porygon
      - warehouse
      - inventory
    oauth_connections:
      o365: graph_sso
    obo_scopes:
      o365:
        - https://graph.microsoft.com/.default
    welcome_message: "Hello! I'm Porygon. How can I help you?"
    debug: false
```

Corresponding environment variables:

```
PORYGON_MICROSOFT_APP_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
PORYGON_MICROSOFT_APP_PASSWORD=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
PORYGON_MICROSOFT_TENANT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
PORYGON_API_KEY=8Z3op0GmLGIhnq2JuPSwTAokV3zGO2Lt2lsKOvGoxvXNXnSDTF6wm
xt6yBLG7X2s
PORYGON_HMAC_SECRET=30ba2f7b5e1fa05d3c1f55327e53262c2e5220d46d8da8673
1628a70e8c3c450
```